

From Hidden Confounders to Honest Answers

A Ground-Up Guide to Our Confounded-POMDP Project

Deep RL Project (Phase 2) — Sharif University of Technology
S. Sayah Varg, M. Shirinbayan, M. Ostadmohammadi

Abstract

This is a self-contained tutorial. If you know what *states*, *actions*, *rewards*, and *MDPs* are, you know enough to read every word of it. Everything else is built up from scratch — analogies first, equations second. We explain the problem our project attacks (learning from decision logs poisoned by a hidden variable), the elegant 2024 theory we implemented to fix it, the place where that theory quietly explodes when you actually run it, the repair we invented, and — being honest — the sobering limits of both the repair and the whole approach. Read it top to bottom like a story; each section sets up the next.

Contents

The 60-Second Version	1
1 The Intuitive Problem: Confounding and Partial Observability	1
1.1 The comfortable world: a normal MDP	1
1.2 Two things go wrong at once	2
1.3 An analogy that makes it click	2
1.4 Why naive evaluation is <i>systematically</i> biased (not just noisy)	2
2 The Anchor’s Core Solution: The Dual-Bridge Idea	2
2.1 The secret ingredient: a “negative control” taken <i>before</i> the decision	3
2.2 What a “bridge function” actually is	3
2.3 From infinite theory to finite matrices: where approximation sneaks in	3
2.4 What our hyperparameter calibration actually did	4
3 The Vanilla Breakdown and Our Innovation	5
3.1 The plan: “pessimism”	5
3.2 The detonation: why vanilla pessimism yields -1755	5
3.3 The innovation: Signal-Projected Pessimism	6
3.4 The honest twist: a heuristic, not a guaranteed cure	6
4 The Empirical Verdict	7
4.1 The math is implemented correctly (the 10^{-15} handshake)	7
4.2 More data $\rightarrow$ smoothly less error	7
4.3 The boundary condition: where it loses to a dumb baseline	8
4.4 The honest bottom line	8
Epilogue	9

The 60-Second Version

We do **offline reinforcement learning**: we have a big log of decisions someone already made (say, doctors treating patients) and we want to judge *new* strategies without ever running them for real. The catch is that the log is **poisoned by a hidden variable** — something the historical decision-maker could see and react to, but which was never written down. That hidden variable makes the naive way of scoring strategies *systematically wrong*, and no amount of extra data fixes it. A 2024 paper (Hong, Qi & Xu) proposed an elegant cure using two objects called **bridge functions** plus a special “before-the-decision” measurement. It was pure theory. **We built it, ran it, and stress-tested it**, found where the math explodes in practice, invented a fix, and honestly mapped where the fix (and the whole method) helps versus where it loses to a dumb baseline.

1 The Intuitive Problem: Confounding and Partial Observability

1.1 The comfortable world: a normal MDP

In a standard **Markov Decision Process (MDP)** you see the full **state** s , take an **action** a , get a **reward** r , and the world moves to s' . “Markov” means the state holds everything relevant: given s and a , the future ignores the past. Here, offline RL is easy in principle: collect a log of (s, a, r, s') , average “after state s and action a , what happened,” and you can score any strategy. The quiet assumption baking this cake is that **you can see the whole state**. That is exactly what breaks in reality.

1.2 Two things go wrong at once

Partial observability. You rarely see the true state s . In an ICU you see *vital signs* O (the observation), but the true physiological state s is hidden — vitals are a blurry shadow of it. A system where you see only observations of a hidden state is a **POMDP** (Partially Observable MDP).

Confounding. Here is the killer. The doctor who generated your log *could see more than you can*: training, intuition, an unrecorded sense of how sick the patient “really” was. They used that hidden information to *choose the treatment*, and the same hidden information also drove the *outcome*. A hidden variable that touches both the action and the outcome is a **confounder**. It is the single most dangerous thing in causal inference.

1.3 An analogy that makes it click

You study a company’s sales logs to decide whether sending a salesperson to a client is worth it. You notice: *visited clients almost always buy*. Naive conclusion: *visiting works, send salespeople to everyone!* But a hidden variable the sales manager saw and you cannot: **which clients were already eager to buy**. The manager sent salespeople *precisely* to the clients about to buy anyway. The visits did not cause the sales — the hidden “eagerness” caused both the visit and the sale. Roll out “visit everyone” and you will be disappointed, because you now visit un-eager clients too. That hidden eagerness is the confounder; it made your log say visiting is magic when it is not. This is exactly our situation, with doctors, treatments, and hidden patient severity.

1.4 Why naive evaluation is *systematically* biased (not just noisy)

This is the point people underestimate. When you score a new strategy naively — treating the observation O as the state and averaging log outcomes — you do not get a *noisy* answer that averages out with more data. You get a **biased** answer that stays wrong *no matter how much data you collect*. The reason: the log’s actions are correlated with the hidden confounder, and so are the outcomes, so your average silently mixes “the effect of the action” with “the effect of the hidden thing that made this action get chosen.” More data just estimates *the wrong number* more precisely.

In our experiments this is concrete. On our controllable test environment (the “toy”), the naive evaluator over-scores policies by +0.14 to +0.33 in value units (8% to 22% relative), and this bias is **flat as N grows**: at $N = 5,000$ and $N = 50,000$ it barely moves. That flatness is the fingerprint of *bias*, not variance — variance shrinks with data, bias does not.

Offline RL breaks when a hidden variable drove both the logged actions and the outcomes. The naive “pretend the observation is the state” evaluator is then systematically biased, and — unlike noise — that bias never washes out with more data. That is the enemy.

2 The Anchor’s Core Solution: The Dual-Bridge Idea

Our project builds on **Hong, Qi & Xu, “Model-based RL for Confounded POMDPs” (ICML 2024)** — the “anchor paper.” It gives a beautiful way to un-poison the log. We build it up in plain language before any equation.

2.1 The secret ingredient: a “negative control” taken *before* the decision

The trick starts with an extra measurement we call O_0 : a reading taken *before the first action is chosen* — a snapshot at the very start.

Intuition. Because O_0 happens *before* the doctor acts, it is a clean “echo” of the hidden state that is **not contaminated by the decision**. It is like a **fingerprint of the hidden state, taken at the door before the crime**. It does not reveal the hidden state directly (it is noisy), but it is correlated with it *and* untainted — exactly what lets us correct for a variable we never actually observe.

In causal-inference language O_0 is a **negative control** (or *proxy*). Its formal job (the paper’s Assumption 3.1) is: *given the true hidden state and the action, O_0 is independent of the later observations, next-states, and rewards*. In words: once you know the hidden truth, the early fingerprint tells you nothing new. All of O_0 ’s value is that it is an untainted window onto the hidden state — nothing sneaky.

2.2 What a “bridge function” actually is

Now the centerpiece: two **bridge functions**, b_R (reward bridge) and b_D (dynamics bridge).

A bridge function is a re-expression of a hidden-state quantity purely in terms of things you can see, built so that its average — viewed through the lens of the negative control O_0 — matches a quantity you can measure from data.

Intuition. You want to know something about the hidden state (e.g. the true reward a policy would earn), but you only see observations. A bridge function is a **corrective lens**: a function of the *observed* variables shaped so that looking at it *through the O_0 window* reproduces the real conditional probabilities in the data. The deep theorem says: if a function of observables reproduces the observable data when viewed through O_0 , then that same function reproduces the *hidden-state* quantity you actually wanted. The O_0 window ties the two together.

There are two bridges because there are two hidden-state quantities to rebuild:

- b_R — **the reward bridge**: recovers the reward model (what rewards a policy would really earn).
- b_D — **the dynamics bridge**: recovers the transition model (how the hidden state would really evolve).

Together they rebuild the *entire* confounded-POMDP machinery from the poisoned log (this is why the method is “model-based” and “dual-bridge”). Chain them and you can evaluate *any* policy cleanly, with confounding subtracted out.

2.3 From infinite theory to finite matrices: where approximation sneaks in

The paper is written for *infinite populations and integral equations*. Reality gives a *finite pile of trajectories*. The gap is where every practical difficulty lives. The bridge’s defining “matching condition” is really a **Fredholm integral equation of the first kind** — “solve for the function b inside an averaging operator”:

$$\underbrace{\mathbb{E}[b(W_t, y) \mid X_t]}_{\text{average of } b \text{ you can see, given the } O_0\text{-window}} = \underbrace{p(y \mid X_t)}_{\text{a probability you can measure}}, \quad (1)$$

for every index y , where $W_t = (A_t, O_t)$ are the observed variables and $X_t = (A_t, H_{t-1}, O_0)$ is the conditioning that includes the negative control.

Intuition. Solving for b means **inverting an averaging operator**. But averaging *smooths things and throws information away*, so un-averaging it is **ill-posed**: tiny noise wiggles in the measured right-hand side get *amplified* into big wiggles in the recovered b . This ill-posedness is the villain of Section 3.

The paper’s estimator, which we implemented, handles this in **two stages**:

- **Stage 1 (learn the lens)**. From part of the data, estimate the averaging operator itself (a *conditional mean embedding*). In our tabular setting this is literally **counting**: build tables of conditional frequencies. Call the result the “lens matrix” Γ .
- **Stage 2 (solve for the bridge)**. From the *other* part of the data, solve the matching equation using the Stage-1 lens. In closed form:

$$\alpha = (G + N_2 \lambda_2 I)^{-1} \hat{p}, \quad G = (\Gamma^\top K_W \Gamma) \odot K_{Y''}, \quad (2)$$

where G is built from the lens and the data, $\hat{p}$ is the measured right-hand side, $\odot$ is an element-wise product, and α are the coefficients that *are* the estimated bridge.

The only genuinely new piece in (2) is the term $+ N_2 \lambda_2 I$: the **regularizer**.

Intuition. Because inverting an ill-posed operator amplifies noise, we deliberately add a little “stiffness” (λ_1 in Stage 1, λ_2 in Stage 2) that stops the solution from chasing noise — at the cost of a little bias. It is the universal **bias–variance knob**: turn λ up for a stable-but-slightly-wrong answer; turn it down for a less-biased but wildly noisy one.

2.4 What our hyperparameter calibration actually did

The paper prescribes *theoretical schedules* like $\lambda \propto N^{-1/2}$, derived for the *infinite-dimensional kernel* version. We implemented the *tabular* version — and hit our first real finding.

*The paper’s theoretical λ schedules do **not** transfer to the tabular setting. Plugging in $\lambda_1 \propto N^{-1/2}$ injected $\sqrt{N_1}$ fake “pseudo-counts” into our count tables, over-shrinking the Stage-1 lens by roughly 80% by the third time-step. The theory’s stiffness was calibrated for a different object and was far too stiff for ours.*

So we did the honest thing: a **grid calibration** — sweep the knobs over a small grid, across three seeds, and *measure* which settings minimize error. The winners:

- **Data split** $N_1 : N_2 = 0.7 : 0.3$ — give Stage 1 (learning the lens over a growing history) more data. This confirmed the paper’s qualitative “Stage 1 needs more” intuition.
- $\lambda_1 = 1/N_1$ — a gentle, parametric-rate stiffness.
- $\lambda_2 = 0.03/\sqrt{N_2}$ — crucially, this decays *slower* than $1/N_2$. It must, because λ_2 ’s job is to **dominate the noise leaking out of Stage 1** into the weakly-identified directions of the bridge. Too fast, and that leaked noise wins.

A subtler lesson that matters in Section 3: the obvious “is my matrix healthy?” check is its **condition number** — and here it is a **liar**. Because of the null-space structure (Section 3), it stays flat and reassuring even as the estimate quietly collapses. The honest health-check is the smallest *signal* eigenvalue $\hat{\sigma}_2$ — the weakest direction that still carries real information. We verified this by degrading O_0 : the true Stage-1 error blew up $10\times$ while the condition number *did not budge*. So all our stability monitoring watches $\hat{\sigma}_2$, not the condition number.

A “bridge” is an observable function whose O_0 -windowed average reproduces the data — and, by a deep theorem, thereby reproduces the hidden-state model you wanted. Estimating it from finite data is a two-stage, ill-posed matrix inversion, tamed by regularizers λ_1, λ_2 whose right values we had to measure, not inherit from the theory.

3 The Vanilla Breakdown and Our Innovation

This is the most interesting part: where the beautiful theory, actually run, **detonates** — and what we did about it.

3.1 The plan: “pessimism”

Once you can estimate the model, you want to *pick a good policy*. But your bridges are noisy, so if you just trust them and pick the best-looking policy, you fall for one that got *lucky* in your noisy estimate. The standard defense is **pessimism**: score a policy by a cautious *lower bound*, and pick the policy whose *cautious* score is highest. This “be paranoid about what you can’t pin down” principle is provably right for offline RL.

The anchor paper draws, around the estimate $\hat{b}$, a **confidence region** — a fuzzy ball of “bridges that fit the data almost as well as $\hat{b}$.” Because the underlying error is exactly quadratic, this region

is a clean **ellipsoid**. The pessimistic value is then

$$V_{\text{low}}(\pi) = \min_{b \in \text{confidence ellipsoid}} V(\pi, b),$$

“the smallest value of policy π you can get as the bridges range over the whole plausible region.” Minimizing a linear value over an ellipsoid has a gorgeous closed form — the **support function**:

$$V_{\text{low}} = V(\hat{b}) - \sqrt{\xi} \|g\|_{H^{-1}}, \quad (3)$$

where ξ is the ellipsoid’s size, g is the value’s gradient, and $\|\cdot\|_{H^{-1}}$ measures length in the ellipsoid’s own stretched geometry. In words: *start at the estimated value, then subtract a penalty for how much the value could wiggle within the plausible region.* Beautiful. So what goes wrong?

3.2 The detonation: why vanilla pessimism yields -1755

Here is the crux, and it is a genuinely subtle geometric fact.

Recall from Section 1 that **the observation is richer than the hidden state** ($|O| > |S|$). In our benchmark, 720 observable “core” states stand in for a 2-valued hidden variable; even in the tiny toy there are 3 observations for 2 hidden states. That mismatch means the data-fitting matrix has a **null space** — directions in “bridge space” the data says *nothing* about. You can move the bridge along a null direction and the data-fit *does not change at all*, because those directions describe patterns in the rich observation with no counterpart in the poor hidden state.

Now look at the ellipsoid. Its “stiffness” in a direction is how much moving there hurts your fit. In the null directions, moving hurts *only* through the tiny regularizer λ_2 . So the ellipsoid is **catastrophically elongated** along the null space — not a round ball, but a **needle** stretching almost infinitely into the directions where the data is blind. Concretely, the pessimism may travel a distance $\|d\| = \sqrt{\xi/\lambda_2}$ along a null direction, and since λ_2 is tiny, that distance is *enormous*.

The vanilla recipe minimizes the value **jointly over all bridge blocks at once**. Let a joint minimization loose on a needle stretching to the horizon in the blind directions, and it *runs to the horizon*: it finds these free, data-invisible directions, pushes the bridges way out along them, and the multi-step value chain amplifies the excursion into nonsense. We measured exactly this:

Ellipsoid size c	0.1	0.3	1.0	3.0	10.0
V_{low} (best policy)	−1.12	−10.4	−61.6	−301.7	−1755

True policy values live between about $+1.5$ and $+2.2$, so a “cautious lower bound” of -1.12 is already *physically impossible*, and -1755 is **deranged**. It is so busy being paranoid about directions the data cannot even see that it produces numbers with no connection to reality and “selects” policies essentially at random.

One-sentence diagnosis: *the paper’s pessimism spends all its paranoia on the empty, data-blind null space instead of on the directions where the data actually has something to say.*

3.3 The innovation: Signal-Projected Pessimism

Once you *see* the problem, the fix writes itself. If the disaster comes from the pessimism wandering into the null space (the empty noise directions), then **do not let it go there**: confine the whole computation to the **signal subspace** — the directions the data actually informs.

How do you find the signal subspace? With the workhorse of applied linear algebra: the **SVD / eigen-decomposition**. Decompose the data’s design matrix into directions ranked by how much real information (signal) they carry. Top directions with healthy eigenvalues are **signal** (the data speaks clearly). Bottom directions with near-zero eigenvalues are the **null space** (the data is silent). An *eigengap* rule draws the line. We then build an orthonormal basis U for the signal directions and run the *same* support-function pessimism, but **restricted to U** :

$$b_{\min} = \hat{b} - \sqrt{\xi} \frac{U H_{\text{sub}}^{-1} U^{\top} g}{\|U^{\top} g\|_{H_{\text{sub}}^{-1}}}, \quad H_{\text{sub}} = U^{\top} H U. \quad (4)$$

Intuition. Project everything onto the signal subspace first, then be pessimistic only within it. The pessimism can no longer escape into the blind directions because we walled them off. The needle becomes a sensible ellipsoid again.

And on the surface it works, beautifully. The projected pessimistic value stays *informative*: for the best policy it is $V_{\text{low}} = 1.35$ (sane, in-range) instead of -1.12 . And it **picks the right policy**: 0.000 **selection regret**, across every random seed. Where vanilla chose garbage, the projected version reliably identifies the truly best policy.

3.4 The honest twist: a heuristic, not a guaranteed cure

Here is the part we refuse to sugar-coat, because being honest here is the most valuable thing the project did.

Walling off the null space *assumes the true bridge lives inside the signal subspace*. We checked that assumption by measuring how much of the true bridge leaks *out* of the signal subspace. The answer: about **21%** of its length (for one bridge, the leaked-out part had length 0.38 while the whole bridge had length 1.80).

That leakage is fatal to the *guarantee*. If the truth is 21% outside the region you search, your “confidence region” **does not contain the truth** — its honest coverage is *zero*. A pessimistic bound is only trustworthy if the truth is in the set you minimize over; ours is not. So $V_{\text{low}} \leq V_{\text{true}}$ held in all our runs *by luck of sign-alignment*, not because the geometry guarantees it.

*There is a fundamental **tension between coverage and informativeness**. The vanilla method covers the truth but is uninformative (blows up to -1755 , picks garbage). Our projected method is informative (sane numbers, 0.000 regret) but does not cover the truth (walls off 21% of it). You cannot have both with this estimator. Signal-Projected Pessimism is an excellent, practically-useful **heuristic** — but it is not a mathematically guaranteed pessimistic bound, and we say so plainly.*

The real intellectual product here is not “we fixed the paper,” but “we found precisely *why* the paper’s beautiful pessimism is unusable as written, we made it usable, and we mapped the exact price of doing so.”

4 The Empirical Verdict

Theory and analogies are nice, but this was fundamentally an *engineering-and-measurement* project. Here is what we actually observed, in order of increasing humility.

4.1 The math is implemented correctly (the 10^{-15} handshake)

Before trusting any result, prove the code computes what the equations say. We used **oracle checks** — comparisons against a ground truth computed exactly in a small environment:

- The value chain matches exact dynamic programming to 4.4×10^{-16} . Feeding the *true* bridges into our evaluator reproduces the exactly-correct value to machine precision (essentially the smallest gap a 64-bit computer can represent). Our tensor contractions *are* the equations, bit for bit.
- The estimator hits the right target to 3.0×10^{-15} : in the infinite-data limit our bridge solver lands on the exact ideal solution.
- Two independent derivations (a “dual” and a “primal” form of Eq. (2)) agree to 2.4×10^{-15} — a strong internal consistency proof.

These 10^{-15} -level agreements are the bedrock: every *interesting* result later is a fact about the *method*, not a bug in the *code*.

Intuition. A humbling aside that captures the whole project’s spirit: while building a comparison baseline, our first version was a perfectly reasonable-looking derivation that the oracle immediately exposed as *systematically wrong* (it gave 1.61 for a policy whose true value is 1.865, and the error did not shrink with data). Only running it against the exact answer caught it. The lesson we kept relearning: **reasoning proposes, the oracle disposes**.

4.2 More data $\rightarrow$ smoothly less error

On the toy, where we can measure bridge-recovery error against the true bridges, the error drops **monotonically at every step** as we add data:

Trajectories N	1,000	5,000	10,000	20,000
Reward-bridge error err_{b_R}	1.360	0.775	0.581	0.459
Dynamics-bridge error err_{b_D}	0.732	0.422	0.312	0.223

Fitting a line on a log–log plot gives decay rates of about $N^{-0.36}$ and $N^{-0.39}$ — steady, honest statistical convergence. The payoff: our de-biased evaluator beats the naive (confounding-blind) baseline in **12 out of 12** test cases on the toy. *Where confounding is strong and identification is clean, the method does exactly what it promised: it removes the bias that more data alone never could.* This is the success story.

4.3 The boundary condition: where it loses to a dumb baseline

Now the humbling and most scientifically important result. Scaling to the full benchmark (720 observable states, a realistic size), the story **reversed**. We ran a comprehensive sweep — five confounding strengths κ from 0 (none) to 1 (maximal), three data sizes up to 60,000, three seeds, four methods — and measured every method against the true policy values.

The naive, confounding-blind baseline **beats** our sophisticated de-biasing method almost everywhere at benchmark scale. The plug-in’s error (0.006–0.12) is comparable to or larger than the structural bias it was built to remove (0.004–0.011).

Intuition. Recall the two enemies: *bias* (from confounding) and *variance* (from finite data). The bridge method kills the *bias* — but killing it requires inverting ill-posed matrices, and **inverting ill-posed matrices amplifies noise**, i.e. it *injects variance*. At benchmark scale each little 2×2 correction is estimated from a handful of samples, so the variance it injects is *bigger than the bias it removes*. You pay more in noise than you gain in bias reduction. The cure is worse than the disease.

The most telling data point: the method’s *worst* performance is at $\kappa = 0$, where there is *no confounding at all* — up to **23–46** $\times$ worse than naive. A correction tool that damages the answer when there is nothing to correct is telling you loudly that its problem is *variance*, not bias.

We tried to fix this honestly. A **James–Stein shrinkage** upgrade (pool each noisy little matrix toward a stable, confounding-aware group average) genuinely helped where it should: under strong confounding it **halved** the error ($0.22 \rightarrow 0.08$) and cut run-to-run variance **6** $\times$. A real, defensible improvement — but *not enough to overturn the boundary*; shrinkage narrows the gap to naive without closing it.

We also built a **model-free** competitor (estimate value functions directly instead of the model). It was worse and *flatter* still — its error *did not improve with more data at all*. The reason is instructive: our negative control O_0 is only a *binary* fingerprint, too little information to pin down a value function over 720 observations. It is **under-identified** — like reconstructing a detailed photograph from a single bit. (Reassuringly, on the toy where identification is clean, this same method *is* consistent — so its benchmark failure is a genuine identification limit, not a bug.)

4.4 The honest bottom line

- **Model-based beats model-free** at benchmark scale (the model-free method is starved by the tiny proxy).
- **But both lose to naive** at realistic sample sizes, because estimation *variance* overwhelms the confounding *bias* they are designed to remove.
- **Shrinkage helps under strong confounding** but does not change the verdict.
- **The method shines exactly where the theory promised** — strong confounding, clean identification, enough data (our toy) — and struggles exactly where practice bites (large observation spaces, weak proxies, limited data).

The through-line of the entire project, in one sentence: **at realistic scale, variance — not identification — is the wall**. The anchor paper solved the identification problem elegantly; what it (understandably, being pure theory) never had to confront is that solving identification by matrix inversion re-introduces a variance problem that can be worse than the bias you started with.

Epilogue: What This Project Taught Us

1. **A gorgeous theory can be silently unusable, and only running it reveals where.** The –1755 blow-up, the null-space geometry, the 21% leak, the benchmark reversal — none are visible in the paper. They appeared only when exact code met exact oracles. Every real finding came from *measurement*, not from thinking harder.
2. **Honesty about a heuristic beats a fake theorem.** We could have called Signal-Projected Pessimism a “repair” and moved on. Instead we measured the 21% leak and reported the coverage/informativeness tension. That negative result is more useful — and more true — than a victory lap.
3. **The bias–variance trade-off is the deepest force in the room.** Confounding is a bias problem; the elegant fix is a variance-injecting inversion; at scale the variance wins. Understanding *that*, and mapping exactly where the crossover happens, is the most valuable thing to hand to whoever builds on this next.

If you read this far, you now understand — from first principles — a genuine research-grade problem in causal reinforcement learning, an elegant proposed solution, exactly how and why it breaks in practice, a clever-but-honestly-limited fix, and the sober empirical reality of when it helps and when it does not. That full arc — the promise, the breakdown, the repair, and the honest verdict — is the real education.